

TracePilot 项目汇报 — 逐页内容详案

基于大纲的详细幻灯片内容设计，共 16 页
每页包含：布局建议 / 标题与正文内容 / 插图建议 / 演讲要点

第 1 页 — 封面

布局建议

全幅背景式封面。主标题居中偏上，副标题居中偏下，团队信息和日期在右下角。

内容文案

主标题（大字）：

TracePilot

副标题（小字）：

Frame-Centric 的 Android 调度辅助系统

底部信息：

Frame-aligned, dependency-aware scheduling assistant for Android interaction workloads

右下角：

TracePilot 团队 · 潘智勇 李松茂 邵晨轩 贺小轩 杨子皓 · 2026 年 6 月

插图建议

- 用一张 Pixel 6a 手机概览图作为背景（半透明覆盖）
- 或使用 eBPF + Perfetto 的 Logo 组合作为装饰元素

演讲要点

简短介绍项目名称和团队。引出核心：我们解决的是 Android 卡顿问题。

第 2 页 — 项目背景与研究问题

布局建议

左半部分"问题"，右半部分"解决方案"的对比布局。

内容文案

主标题： Android 卡顿：从何而来，如何定位？

左侧：核心问题

- Android 交互卡顿（Jank）严重影响用户体验
- 根因往往是 **跨进程等待链**：
UI thread → RenderThread → Binder → system_server → SurfaceFlinger
↑ _____ 卡顿根源在此 _____ ↑
- 传统单进程分析方法无法有效定位

右侧：为什么 PID-Centric 不可行

问题	说明
PID 不稳定	同一 App 每次启动 PID 不同，还可被复用
卡顿不是单个 PID 的事	jank 根因是跨进程等待链，需全局视角
eBPF 只观测内核事件	无法直接回答"用户在经历什么"

中间对比结论框（大字，高亮）：

- ✗ PID 视角：看到 pid=1234 进程 CPU 占用高 → 无法定位根因
- ✓ Frame 视角：看到 f32 帧内 SurfaceFlinger 被 Binder 调用阻塞 12ms → 精准定位

底部核心路径：

FrameTimeline 定义问题 → eBPF 提供原因 → Graph 找关键路径 → Hint Engine 做受控干预

插图建议

- 左侧展示跨进程等待链（UI → RenderThread → Binder → system_server → SurfaceFlinger），在 Binder 处用红色高亮标注"Jank"
- 右侧用两张对比图：PID 视角（散乱的进程号）vs Frame 视角（以帧为窗口的整齐排列）
- 中间大字对比结论用鲜明底色突出

演讲要点

重点讲清楚两个认知转变：(1)为什么传统的用 PID 看问题不行；(2)为什么必须以帧为对齐单位。这一页是整个汇报的论证起点，务必让听众理解。

插图建议

- 一个简单的时序图：展示一帧从 UI 线程开始，经过 RenderThread → Binder → SurfaceFlinger 的流程，标注"此处卡顿"
- 或使用两张对比图："PID 视角(散乱)" vs "Frame 视角(对齐)"

演讲要点

重点讲清楚"为什么传统的用 PID 看问题的方式不行"和"为什么要以帧为对齐单位"这两个认知转变。

第 3 页 — 项目历程与开发路线图

布局建议

上半部分用时间线/阶段图，下半部分用文本框展示迭代反馈。

内容文案

主标题：项目开发路线图

阶段时间线（从左到右的箭头图）：

前期调研(第1个月) → Step 1(基础能力) → Step 2(增强能力) → Step 3(深化) → Step 3+(展望)				
4 份调研报告 可行性分析 技术路线确定	Perfetto 帧提取 eBPF sched 采集 身份解析+Hint	Binder/Futex 图 Jank 分类器 CPU 频率+视频扩展	Thermal 深化 Inference 证据链 Multi-session 对比	sched_ext Learned Policy Cuttlefish
✓ 已完成	✓ 已完成	✓ 已完成	✓ 已完成	未做

底部：老师指导与迭代

刑凯老师改进建议：从"全量采集"转向"场景驱动"，聚焦 1-2 个核心场景，建立数据质检流程，做好特征语义映射

➡ 项目据此调整方向 → 收敛到**页面切换 + 视频浏览**两大核心场景

插图建议

- 水平时间轴，按时间顺序排列里程碑
- 每个阶段用不同颜色标记，已完成的打勾，未做的用虚线框

演讲要点

让听众了解项目的整体推进节奏。强调"老师反馈 → 调整方向"的迭代过程，展示项目从宽泛到聚焦的演进。

第 4 页 — 前期调研成果

布局建议

上半部分四张卡片展示四份报告，下半部分三条核心结论。

内容文案

主标题： 前期调研 — 技术路线确立（第 1 个月）

四份报告卡片：

📄 调研报告	📄 可行性报告
Android 16 eBPF 预测调度方案	eBPF 数据采集 7 层模型设计
LLM/自研时序模型 + 调频指令注入内核	统一上下文层 → 可扩展观测层

📄 补充调研报告	📄 TracePilot 调研扩展
系统行为观测扩展（网络、传感器）	Page Turning、Feed Scroll 场景分析
高频场景扩展（支付、游戏等）	行为特征分析方法论
关键/非关键线程识别方法论	

核心结论（三列强调）：

- 1. eBPF 适合承担内核态行为观测，最关键是调度事件 + Binder + Futex + CPU 频率
- 2. 一期聚焦页面切换，不承诺网络栈/文件系统/IRQ 等扩展观测
- 3. 采用 **Frame-Centric** 而非 **PID-Centric** 的观测视角

插图建议

- 四份报告的缩略封面排列展示
- 7 层模型的小示意图（从底层统一上下文到上层可扩展观测）

演讲要点

简要带过调研阶段，说明做了充分的可行性分析才确定了技术路线。重点强调"7 层数据模型"的设计思路和"聚焦优先"的策略。

第 5 页 — 技术栈总览

布局建议

中间一个大图展示整体架构，四周标注各技术的位置和作用。

内容文案

主标题： 技术栈与系统架构

架构图（分层架构）：

分析层 (Python + sklearn)
特征提取 · 图构建 · Jank 分类 · 对比分析 · 报告
用户态加载层 (C loader)
eBPF 加载 · ringbuf 读取 · 图构建 · Hint Engine
帧标定层 (Perfetto)
trace_processor_shell + SQL → frames.txt
内核采集层 (eBPF)
kprobe + tracepoint + ringbuf → events.bin
硬件层 (Pixel 6a)
Android 14 / Magisk root / ARM64

右侧技术详情表：

技术	版本/工具
eBPF	C 编写 BPF 程序，clang 交叉编译
Perfetto	trace_processor_shell SQL 查询
Docker + NDK r26b	Ubuntu 22.04 编译环境封装
Python + sklearn	决策树分类 LeaveOneOut 验证
C (loader)	身份解析 + 图构建 + 推断引擎

插图建议

- 分层架构图是这张的核心，一定要画清楚
- 每个层次可以用不同的颜色底色

演讲要点

这张是全局架构 overview，让听众尽快建立整体认知。"我们从 Pixel 6a 硬件开始，通过 eBPF 采集内核事件，Perfetto 做帧标定，C loader 做解析和构图，Python 做分析和训练"。时间不够可以快速过。

第 6 页 — eBPF 探针设计

布局建议

上方表格列出探针，下方展示编译部署流程的流程图。

内容文案

主标题：eBPF 探针 — 内核事件采集

推导逻辑（为什么选这些探针？）：

用户感知卡顿 → 帧掉 → 线程没跑 → 三种可能：**调度排队 / 等 Binder / 等锁**
→ 因此必须同时采集 sched_switch + binder + futex
→ 再加 cpu_frequency + thermal 解释环境因素

探针一览表：

探针	挂载点	采集内容	用途
sched_switch	tracepoint	线程切换 prev/next TID、运行时长、runnable delay	计算就绪等待延迟
sched_wakeup	tracepoint	唤醒延迟	wakeup-to-run latency
binder_transaction	kprobe ⚠️	Binder IPC 调用发起/接收	跨进程依赖分析
futex wait/wake	tracepoint	锁等待	同步阻塞识别
cpu_frequency	tracepoint	CPU 频率变化	大小核调频分析
thermal_temperature	tracepoint	温控温度	降频归因

⚠️ binder 用 kprobe 而非 tracepoint：Android GKI 内核未暴露 binder tracepoint

编译部署流程：

Docker(NDK r26b + clang) → 交叉编译 → tracepilot.bpf.o + tracepilot-aarch64
↓ adb push
Pixel 6a: bpf() 系统调用加载
↓
ringbuf 输出 events.bin (450~690MB)

插图建议

- 在探针表上方，用一个简短的推导链（用户感知卡顿 → 帧掉 → 线程没跑 → 三种可能 → 对应探针），用箭头串联
- 每个探针用小图标表示（绿色=调度、蓝色=IPC、橙色=锁、红色=电源）
- binder 的 kprobe 挂载点用 ⚠️ 或红色标记，突显与 tracepoint 的不同

演讲要点

"为什么选这 6 个探针？因为 Android 卡顿只有三种可能原因：线程没被调度、在等 Binder、在等锁。所以我们必须同时采集这三种信号，再加 CPU 频率和温度解释环境。注意 binder 用的 kprobe 而非 tracepoint——Android 内核没暴露 binder 的 tracepoint。"

第 7 页 — 覆盖的四大场景

布局建议

四列卡片布局 + 底部的补充文本框。

内容文案

主标题： 四大采集场景覆盖

四个场景卡片：

页面切换 (基础版)	视频浏览 (增强版)	信息流滚动 (Chrome)	相机场景 (Camera)
QQ 690MB events IRQ/softirq 辅助分析	微信/抖音 451MB events Binder/Futex Jank分类 Hint Engine	Chrome 261万/34s 秒级聚合 34线程级汇总 ftrace补充	Google Cam 全自动Pipeline 编译→部署→ 采集→分析→ 报告

底部补充：QQ 行为特征分析

- 采集 behavior_features.csv (578 行)，按秒级窗口 + 包名聚合
- QQ 主包 com.tencent.mobileqq 的突发行为模式 (P90 阈值 = 36 事件/秒)
- 识别系统侧并发干扰 (系统服务、后台应用对主场景的影响)

插图建议

- 每个场景配一个手机截图或 Logo
- 不同场景用不同颜色区分

演讲要点

"我们覆盖了 4 个典型交互场景。页面切换是核心，视频浏览增加了解码分析维度，信息流滚动验证了高事件率下的可行性，相机场景则实现了全自动 Pipeline。"

第 8 页 — Frame-Centric 对齐算法

布局建议

中央展示对齐流程图，上下分别描述问题和关键指标。

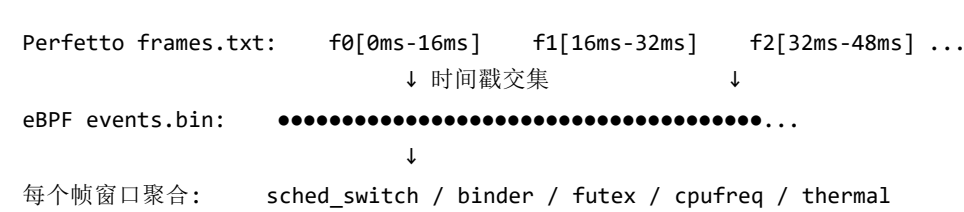
内容文案

主标题： 核心对齐算法：eBPF 事件如何与帧对齐？

问题陈述：

Perfetto 帧时间线提供帧边界（ expected_start / expected_end / actual_end ），但 eBPF 事件是独立采集的纳秒级内核事件流。两者必须对齐才能回答"这个 jank 帧内发生了哪些内核事件"。

对齐流程图：



- Jank 帧独立开窗
- 超出窗口的事件通过"依赖继承机制"保留（如跨帧的 binder 调用）
- 帧内聚合 → jank 级别判定

关键指标：

- 帧内 runnable delay 总和
- Binder 调用深度
- Futex 竞争强度

插图建议

- 对齐流程图中，Perfetto 帧用不同颜色条表示，eBPF 事件用圆点，对齐区域用高亮

演讲要点

"这是整个项目最核心的对齐问题。Perfetto 告诉我们哪些帧卡了，eBPF 告诉我们内核里发生了什么——但它们是两个独立的数据流。我们的对齐算法通过时间戳交集把它们关联起来。"

第 9 页 — 图构建算法：从原始事件到依赖图

布局建议

上半部分节点构建 + 边构建，下半部分公式 + 可视化说明。

内容文案

主标题： 图构建 — 从原始事件到依赖关键路径

节点构建： 每个节点 = 一条线程

节点属性	说明
id (TID)	线程 ID
comm	线程名
role (角色)	classify_thread() 自动识别的 12 类角色
frame_window_overlap	帧窗口重叠度

角色识别算法：

角色	判定依据	颜色
UI Thread	TID==target_pid 或 comm 以"com."开头	
RenderThread	comm 含"renderthread"	
SurfaceFlinger	comm 含"surfaceflinger"	
Binder RPC	comm 含"binder"	
GPU Worker	comm 含"gpu"/"gl"	
KernelWorker	comm 含"kworker"/"swapper"	

边构建： 什么是依赖？

边类型	来源事件	构建逻辑
BINDER_CALL	binder_transaction	TX→RX
FUTEX_WAIT	futex	同一 uaddr wait→wake
SCHED_DEPENDENCY	sched_wakeup	wakee→waker
DECODE_DEPENDENCY	视频帧	解码→渲染
RESOURCE_STALL	cpu_frequency	CPU 资源不足

CriticalScore 公式（核心算法）：

$$CriticalScore(T) = \alpha \times CriticalPosition(T) + \beta \times \frac{RunnableDelay(T)}{TotalDelay} + \gamma \times ConnectionDegree(T)$$

其中 $\alpha=0.4, \beta=0.4, \gamma=0.2$ （可调）

直觉解释（为什么是这三个维度？）：

维度	含义	直觉
CriticalPosition	线程在关键路径深度	越靠近帧渲染末端越关键
RunnableDelayShare	该线程延迟 / 总延迟	占帧内浪费时间的比例越大越关键
ConnectionDegree	Binder/Futex 连接数	影响其他线程越多越关键

可视化方法：

- BFS 分层布局 + 同层按 CriticalScore 降序
- 三层子图：Binder / Futex / 关键路径
- 颜色编码区分角色

插图建议

- 左侧展示一个简化的小图例（3-4 个节点 + 边）
- 右侧展示实际报告的 SVG 图缩略（Binder 图或 Futex 图）

演讲要点

"这是项目的技术核心。我们把每条线程变成一个图节点，把 Binder 调用、Futex 锁等待变成边。然后通过 CriticalScore 公式给每个节点打分——分数越高，该线程越可能是卡顿的根因。\$\$

第 10 页 — 核心分析成果（页面切换 + 视频浏览）

布局建议

上方表格展示三次采集对照，下方展示根因推断和关键发现。

内容文案

主标题： 核心分析成果 — 三次采集对照

三次采集对照表：

维度	页面切换 Run 1	页面切换 Run 2	视频浏览
总帧 / Jank	2171 / 1575	1271 / 1228	2141 / 1524
Jank 率	72.5%	96.6%	71.2%

维度	页面切换 Run 1	页面切换 Run 2	视频浏览
VD 帧	0	0	561
图规模	6968节点/5234边	4799节点/3106边	6622节点/4992边
边类型	BINDER=52, FUTEX=2371	BINDER=53, FUTEX=1770	BINDER=407, FUTEX=1436
调频抑制比	0.00	0.47	1.00
最高温度	31.4°C	41.5°C	52.3°C

根因推断：

场景	根因	推荐 Hint	置信度
Run 1	RUNNABLE_DELAY	PROTECT_UI_CHAIN → surfaceflinger	0.9999
Run 2	RUNNABLE_DELAY	UCLAMP_MIN_TEMPORARY	0.9999
视频	RUNNABLE_DELAY	BOOST_THREAD	0.9999

三条关键发现：

- 1. Run 2 温度升高+降频（throttle=0.47），推荐策略从 PROTECT 升级为 **UCLAMP**
- 2. 视频浏览温度 **52.3°C**，完全降频（throttle=1.00），典型的温控场景
- 3. Top-5 嫌疑线程含 rcuop、kworker 等系统线程，说明卡顿涉及 **系统级资源竞争**

插图建议

- 三个场景的数据对比可以用柱状图或热力图展示
- SVG 图缩略（Binder 图、Futex 图、关键路径图）

演讲要点

"这是我们最核心的实验结果。三次采集覆盖了不同的温度和降频条件，Jank 率从 72% 到 97% 不等。注意 Run 2 由于温度升高，Hint Engine 自动把推荐策略从 PROTECT 升级成了 UCLAMP——这体现了我们的 Hint Engine 能根据环境条件自适应调整。"

第 11 页 — Step 1：基础能力实现

布局建议

左侧模块列表，右侧 Hint Engine 安全机制详解。

内容文案

主标题： Step 1 — 基础能力实现

左列：已完成模块

#	功能	实现文件
1	Perfetto 帧提取	frame_query.sql
2	eBPF sched 采集	BPF + events.bin v3
3	身份解析	identity.c
4	Frame window delay	Phase 5b 用户态重算
5	角色识别	classify_thread() 12 类
6	Top-K 关键线程	-G -k N
7	Hint Engine	hint_engine.c

右列：Hint Engine 安全机制

机制	说明
TTL	每个 hint 默认 16ms 过期，超时自动回滚
自旋保护	5ms 内连续 3 次 on CPU 无 sleep → 撤销 hint
线程黑名单	kworker / rcuop / swapper 等不可干预
置信度阈值	>0.7 才提交，<0.5 只记录不执行
audit 日志	每条 hint 的完整操作记录

插图建议

- 模块列表用带编号的方块图
- 安全机制可以用"防护盾"示意图

演讲要点

"Step 1 搭建了整个系统的地基：能从帧中提取信息、识别线程角色、构建依赖图。特别介绍 Hint Engine 的安全设计——因为我们要做的是调度干预，安全是第一优先级，所以设计了 TTL、自旋保护、黑名单等多层安全机制。"

第 12 页 — Step 2：增强能力实现

布局建议

四个分块：Binder/Futex 图、Jank 分类器、Camera 延迟分解、聚合策略对比。

内容文案

主标题： Step 2 — 增强能力实现

① Binder / Futex 图分析

- Binder 图 → 跨进程 IPC 依赖关系（ graph_binder.svg ）
- Futex 图 → 锁竞争热点（ graph_futex.svg ）
- 关键路径图 → 全链路瓶颈（ graph_critical.svg ）
- SVG 自动导出，颜色编码角色

② Jank 分类器（决策树）

- 6 维特征： runnable_delay / binder_centrality / futex_wait / thermal_throttle / decode_late / system_irq
- 自动标注 → 可疑帧筛选 → 人工复核 → 模型训练
- 评估： LeaveOneOut 交叉验证 + confusion matrix
- 导出为 C 头文件 learned_model.h ， 嵌入 loader

③ Camera 延迟分解方法

总阻塞时间 = 调度竞争(*RunnableDelay*) + *Binder IPC* 等待 + *Futex* 锁等待

- 角色识别 → DAG 关键路径 → 根因归因 → 安全调优生成

④ 两种聚合策略对比

策略	适用场景	压缩效果
帧级窗口	页面/视频（需帧精确）	~2000 行
秒级窗口	信息流（宏观趋势）	~35 行

插图建议

- SVG 图缩略展示图的视觉效果
- 决策树的简化示意图
- 延迟分解的饼图/柱状图

演讲要点

"Step 2 是能力的增强。图分析让我们能可视化 Binder 瓶颈和锁竞争；决策树分类器实现了卡顿根因的自动化识别；Camera 场景的延迟分解方法将阻塞时间拆解为三大组成部分，定位精度最高。"

第 13 页 — Step 3：深化与对比

布局建议

四块内容：Thermal 分析、Multi-session 对比、Cross-scenario 验证、Inference 证据链。

内容文案

主标题： Step 3 — 深化与对比分析

① Thermal 深化

- 联动 thermal_temperature + cpu_frequency 计算 freq_throttle_ratio（降频抑制比）
 - 0.0 = 无降频，1.0 = 完全限频
- 实测：视频浏览 **1.00**（52.3°C） / 页面切换 Run 2 **0.47**（41.5°C）
- 温控降频被识别为独立 Jank 根因类别 THERMAL_THROTTLE

② Multi-session 对比

- --compare-dir 自动生成 compare_report.json
- 对比维度：Jank 率、根因分布、图规模、温度、调频抑制比
- 关键发现：
 - Jank 率差异（72.5%→96.6%→71.2%）与温控降频直接相关
 - Run 1 无温控但 Jank 率 72.5% → 调度竞争本身就显著
 - 视频温控 1.00 但 Jank 率与 Run 1 相近 → 视频延迟容忍度可能更高

③ Cross-scenario 分类器验证

- 页面切换（6 维）+ 视频（7 维，增加 decode_late）
- 验证了 **decode_late 在视频场景的必要性**：缺少则误归因为 RUNNABLE_DELAY
- Perplexity 评估输出质量

④ Inference Engine 证据链

信号	来源	权重范围
runnable_delay	sched_switch	0.0~1.0
binder_centrality	binder_transaction	0.0~1.0
futex_wait	futex	0.0~1.0
thermal_throttle	thermal + cpufreq	0.0~1.0
decode_late	视频解码	0.0~1.0
system_irq	irq/softirq	0.0~1.0

多信号加权融合 → hypothesis + confidence → hint 映射

插图建议

- 温度变化折线图
- 三次采集的对比柱状图
- 证据链的流程图

演讲要点

"Step 3 是研究的深化。Thermal 分析让我们量化了温控降频对卡顿的影响；Multi-session 对比回答了一个有趣的问题：页面切换 Run 1 无温控但 Jank 率 72.5%，说明调度竞争本身就很严重，温度只是雪上加霜。Cross-scenario 验证确认了视频场景需要额外的 decode 维度。"

第 14 页 — 自动化与脚本工具

布局建议

两个表格分别展示两个主要场景的脚本工具。

内容文案

主标题： 自动化工具链

页面切换-视频浏览增强版脚本：

脚本	功能
deploy.sh / deploy.ps1	一键部署到 Pixel 6a 并采集
frame_query.sql	Perfetto SQL 查询帧信息
thermal_query.sql	温度数据提取
graph_features.py	图拓扑边分布特征提取
trace_features.py	从 trace 中提取时序特征
trace_label.py	基于规则的自动标签生成
auto_label.py	启发式自动标注 Jank 根因
label_jank.py	人工标注辅助
suspect_frames.py	筛选可疑帧
train_jank_model.py	决策树训练 + C 头文件导出
export_step2_graphs.py	图可视化导出
render_graph_svg.py	SVG 渲染

相机场景脚本：

脚本	功能
auto_run.py	全自动：编译→部署→采集→拉取→分析→报告
analyze_delays.py	延迟聚合 + Binder 配对 + Futex 统计
critical_path.py	DAG 关键路径 + 评分
root_cause.py	根因归因
safe_hint_engine.py	安全调优配置 + shell 脚本
generate_report.py	生成 MD 报告

插图建议

- 以"工具箱"的视觉风格展示，每个脚本是一个工具图标
- 或按 Pipeline 流程标注每个脚本的位置

演讲要点

"我们开发了 18 个脚本工具，覆盖了从部署采集到分析报告的全流程。注意部署脚本同时支持 Linux bash 和 Windows PowerShell，适配不同开发环境。"

第 15 页 — 总结与展望

布局建议

上半部分 9 项成果打勾列表，下半部分 5 个扩展方向。

内容文案

主标题： 总结与展望

已完成成果：

#	成果
✓	完整数据采集 Pipeline：eBPF + Perfetto 覆盖四大场景
✓	身份解析与图构建：帧窗口内的依赖关键路径图
✓	多维度特征提取：6 维特征，涵盖调度、IPC、锁、温控、解码、中断
✓	自动标注 + 决策树分类：端到端 Jank 根因分类 Pipeline
✓	Hint Engine：安全的用户态 hint 推荐（BOOST / UCLAMP / PROTECT）

#	成果
✓	多 Session 对比：页面切换 ×2 + 视频浏览对比分析
✓	信息流滚动补充分析：线程分类 + 评分 + ftrace 融合
✓	自动化部署与分析：一键式脚本，从编译到报告全自动
✓	项目文档：4 份调研报告 + 5 次会议记录 + 7 份分析报告

下一步扩展方向：

方向	说明
▢ sched_ext	可编程调度器，更灵活的内核调度注入
▢ Learned Policy	基于强化学习的调度策略学习
▢ 模型增强	引入大语言模型进行序列预测
▢ Cuttlefish	在虚拟化 Android 环境进行更多实验
▢ 多场景扩展	游戏、相机深化、网络场景

插图建议

- 成果列表用粗体打勾符号
- 展望方向用虚线箭头 + "下一步"标识

演讲要点

"目前项目已完成从数据采集到 Hint 输出的全链路。下一步方向中，sched_ext 是最值得关注的——它是 Linux 6.12+ 的可编程调度器框架，如果能结合我们的 eBPF 观测和 Hint 经验，可以在不修改内核的情况下实现更灵活的调度策略。"

第 16 页 — 附录：关键技术指标

布局建议

大表格一页展示。

内容文案

主标题： 附录：关键技术指标

指标	页面切换 Run 1	页面切换 Run 2	视频浏览	信息流滚动
采集时长	—	—	—	34.2s

指标	页面切换 Run 1	页面切换 Run 2	视频浏览	信息流滚动
原始事件数	~870万	~580万	~500万	261万
events.bin 体积	690 MB	459 MB	451 MB	—
图节点	6968	4799	6622	—
图边	5234	3106	4992	—
人工标注帧	—	—	—	1575+ 帧
Jank 率	72.5%	96.6%	71.2%	—

底部备注：

- 数据日期：2026 年 6 月
- 设备：Pixel 6a (Android 14, Magisk root)
- 采集时长：各场景 30~34 秒
- 全部实验结果存放于 TracePilot/ebpf/ebpf_data/ 目录

插图建议

- 简洁的大表格，可用渐变色突出关键数字

演讲要点

"附录供参考，展示各项关键技术指标。单次采集最高产生 870 万事件 / 690MB 数据，图构建最高达 6968 个节点。信息流滚动 34 秒即产生 261 万事件，说明高交互场景下的数据量是很大的。"

完整 16 页内容编写完毕。每页均包含布局建议、文案内容、插图建议、演讲要点。可根据汇报时间长短选择性使用，核心推荐重点精讲第 2、5、8~10、13 页。